

DỰ ÁN HỌC MÁY TỪ ĐẦU ĐẾN CUỐI

Chương 4 - Huấn luyện Mô hình (Training Models)

TS. Trần Thành Thắng

Đại học Đông Á

August 11, 2026

Nội dung Chương trình

1. Hồi quy tuyến tính & Phương trình chuẩn tắc
2. Hạ Gradient (Gradient Descent)
3. Hồi quy đa thức, Đường cong học tập & Chính quy hóa
4. Hồi quy Logistic & Softmax

- Hiểu rõ cách thức hoạt động bên trong của các mô hình học máy thay vì coi chúng là "hộp đen".
- Tìm hiểu và áp dụng các phương pháp giải Hồi quy tuyến tính (Phương trình chuẩn tắc, SVD, Gradient Descent).
- Phân biệt được các loại Gradient Descent (Batch, Stochastic, Mini-batch) và ảnh hưởng của Tốc độ học (Learning Rate).
- Hiểu cách xử lý dữ liệu phi tuyến tính bằng Hồi quy đa thức.
- Phát hiện Quá khớp (Overfitting) thông qua Đường cong học tập và các kỹ thuật Chính quy hóa (Ridge, Lasso, Elastic Net, Early Stopping).
- Tìm hiểu Hồi quy Logistic cho phân loại nhị phân và Hồi quy Softmax cho đa lớp.

1. Hồi quy tuyến tính & Phương trình chuẩn tắc

Hồi quy tuyến tính (Linear Regression)

- Hồi quy tuyến tính là một trong những mô hình đơn giản nhất. Mô hình sẽ đưa ra dự đoán bằng cách tính tổng trọng số của các đặc trưng đầu vào, cộng với một hằng số gọi là số hạng độ lệch (bias term) hoặc hệ số chặn (intercept term).
- Mô hình dự đoán hồi quy tuyến tính (Dạng toán học):**

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n \quad (1)$$

- Dạng Vector (Rút gọn):**

$$\hat{y} = h_{\theta}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x} \quad (2)$$

- Trong đó: $\boldsymbol{\theta}$ là vector tham số mô hình, $\mathbf{x}$ là vector đặc trưng của trường hợp.

Hàm chi phí MSE (Mean Squared Error)

- Để huấn luyện mô hình Hồi quy tuyến tính, chúng ta cần tìm giá trị cho vector tham số θ nhằm tối thiểu hóa sai số giữa giá trị dự đoán và giá trị thực tế.
- Thước đo phổ biến nhất là Hàm chi phí Lỗi Bình phương Trung bình (MSE - Mean Squared Error).
- **Công thức MSE cho mô hình hồi quy tuyến tính:**

$$MSE(\mathbf{X}, h_{\theta}) = \frac{1}{m} \sum_{i=1}^m \left(\theta^{\top} \mathbf{x}^{(i)} - y^{(i)} \right)^2 \quad (3)$$

- Mục tiêu của quá trình huấn luyện là tìm θ để MSE đạt giá trị nhỏ nhất.

Phương trình chuẩn tắc (Normal Equation)

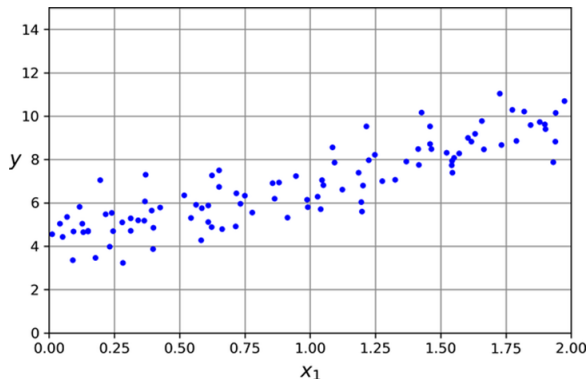
- Để tìm giá trị θ làm tối thiểu hóa hàm chi phí, có một nghiệm dạng đóng (closed-form solution) – một phương trình toán học cho ra kết quả trực tiếp. Nó được gọi là Phương trình chuẩn tắc.
- **Công thức Phương trình chuẩn tắc:**

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (4)$$

- $\hat{\theta}$ là giá trị θ làm giảm thiểu hàm chi phí.
- $\mathbf{y}$ là vector của các giá trị mục tiêu (từ $y^{(1)}$ đến $y^{(m)}$).

Minh họa: Tập dữ liệu tuyến tính ngẫu nhiên

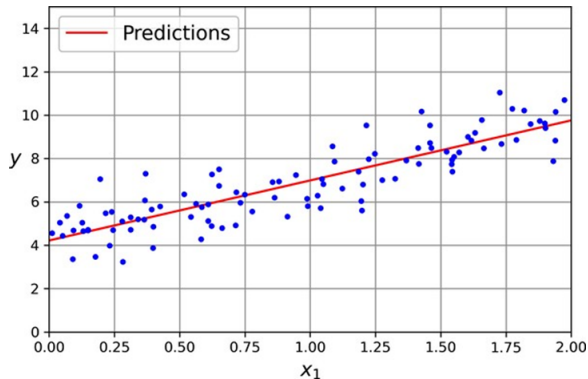
- Để kiểm tra Phương trình chuẩn tắc, chúng ta có thể tạo một số dữ liệu nhìn có vẻ tuyến tính một cách ngẫu nhiên.
- Công thức sử dụng để tạo dữ liệu:
 $y = 4 + 3x + \text{nhiều Gauss.}$
- Biểu đồ bên cạnh cho thấy các điểm dữ liệu được tạo ra. Dữ liệu này sẽ được dùng để khớp mô hình.



Hình 4-1. Dữ liệu tuyến tính được tạo ngẫu nhiên

Minh họa: Dự đoán của Hồi quy tuyến tính

- Sau khi tính toán được $\hat{\theta}$ từ Phương trình chuẩn tắc, chúng ta có thể vẽ đường thẳng dự đoán.
- Hình bên cạnh cho thấy các dự đoán của mô hình (đường thẳng màu đỏ) khớp với tập dữ liệu ngẫu nhiên của chúng ta như thế nào.
- Đường thẳng này đi qua trung tâm của các điểm dữ liệu, cho thấy mô hình đã học được xu hướng chung của dữ liệu.



Hình 4-2. Dự đoán của mô hình hồi quy tuyến tính

Thực hiện với Scikit-Learn và Singular Value Decomposition

- Trong thực tế, Scikit-Learn sử dụng **Singular Value Decomposition (SVD)** thay vì tính trực tiếp Phương trình chuẩn tắc.
- **Lý do:** Ma trận $\mathbf{X}^T \mathbf{X}$ có thể không khả nghịch (chẳng hạn khi số lượng đặc trưng lớn hơn số lượng trường hợp huấn luyện). Phương pháp SVD luôn xử lý được các trường hợp này bằng ma trận giả nghịch đảo (pseudoinverse).
- Đoạn mã Scikit-Learn rất đơn giản:

```
lin_reg = LinearRegression()  
lin_reg.fit(X, y)
```

- Cả phương trình chuẩn tắc và cách tiếp cận SVD đều trở nên **rất chậm** khi số lượng đặc trưng tăng lên đáng kể.
- Độ phức tạp tính toán của việc đảo ngược ma trận (như Phương trình chuẩn tắc) là khoảng $\mathcal{O}(n^{2.4})$ đến $\mathcal{O}(n^3)$.
- Phương pháp SVD được Scikit-Learn sử dụng là khoảng $\mathcal{O}(n^2)$. Tức là, nếu tăng gấp đôi số lượng đặc trưng, thời gian tính toán tăng gấp 4 lần.
- **Ưu điểm:** Cả hai phương pháp này lại là tuyến tính đối với số lượng trường hợp huấn luyện (độ phức tạp $\mathcal{O}(m)$). Chúng xử lý rất tốt tập huấn luyện lớn miễn là nó vừa với bộ nhớ.

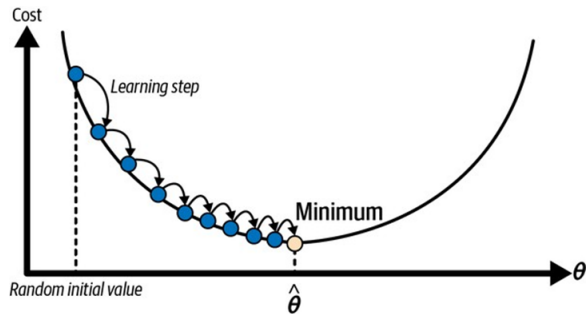
2. Hạ Gradient (Gradient Descent)

Gradient Descent là gì?

- **Gradient Descent (Hạ Gradient)** là một thuật toán tối ưu hóa chung có khả năng tìm ra giải pháp tối ưu cho nhiều loại vấn đề.
- Ý tưởng chung: Tinh chỉnh các tham số theo vòng lặp để giảm thiểu hàm chi phí.
- Ví dụ trực quan: Tưởng tượng bạn bị lạc trên núi trong một màn sương mù dày đặc, và bạn chỉ có thể cảm nhận độ dốc của mặt đất dưới chân mình. Chiến lược tốt nhất để xuống đáy thung lũng là bước xuống dốc theo hướng dốc nhất.
- Nó đo lường gradient (đạo hàm) cục bộ của hàm lỗi so với vector tham số θ , và đi theo hướng giảm của gradient cho đến khi gradient bằng 0.

Minh họa: Nguyên lý hoạt động của Gradient Descent

- Kích thước bước nhảy được xác định bởi siêu tham số **tốc độ học (learning rate)**.
- Bắt đầu với một giá trị θ ngẫu nhiên, thuật toán dần dần tiến gần đến điểm cực tiểu.
- Hình bên minh họa sự mô phỏng quá trình này: các bước đi dần hội tụ tại cực tiểu toàn cục.

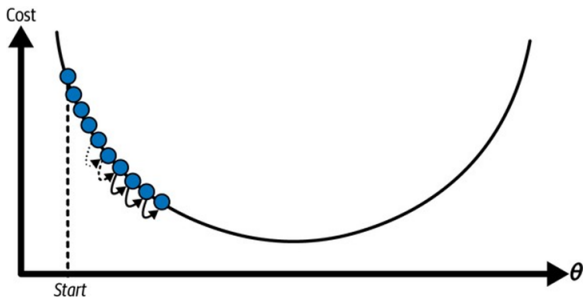


Hình 4-3. Gradient Descent

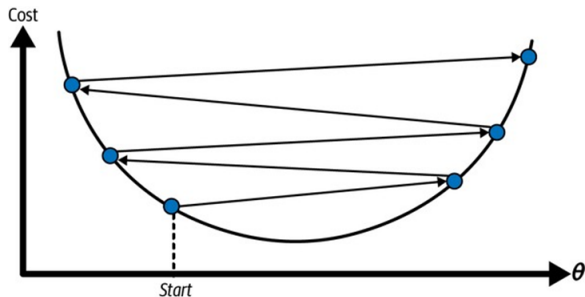
Phân tích ảnh hưởng của Tốc độ học (Learning Rate)

- Tốc độ học (Learning Rate - η) là một siêu tham số quan trọng nhất của thuật toán Gradient Descent.
- **Tốc độ học quá nhỏ:**
 - Thuật toán sẽ phải trải qua rất nhiều vòng lặp để hội tụ.
 - Mất rất nhiều thời gian.
- **Tốc độ học quá cao:**
 - Bạn có thể "nhảy cóc" qua thung lũng và kết thúc ở bên kia, thậm chí cao hơn trước.
 - Làm cho thuật toán phân kỳ (diverge), giá trị hàm chi phí ngày càng tăng và không bao giờ tìm được điểm tối ưu.

Minh họa: Tốc độ học quá nhỏ và quá lớn



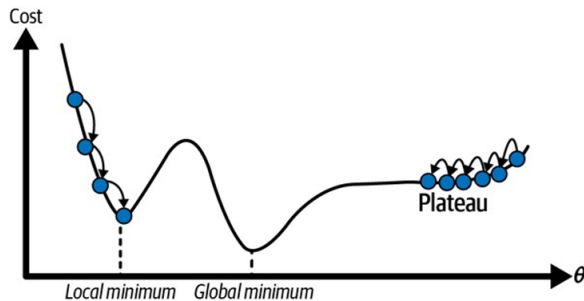
Hình 4-4. Tốc độ học quá nhỏ



Hình 4-5. Tốc độ học quá lớn

Các cạm bẫy của Gradient Descent

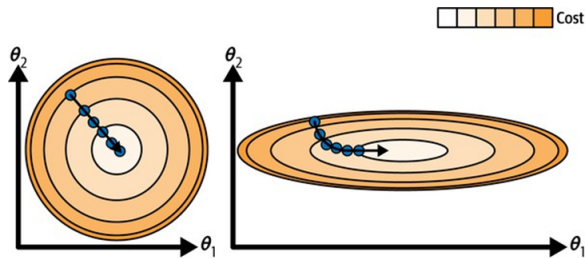
- Không phải tất cả các hàm chi phí đều trông giống như những cái bát hình parabol hoàn hảo.
- **Cực tiểu cục bộ (Local Minimum):** Nếu bạn bắt đầu từ một điểm khởi tạo ngẫu nhiên, thuật toán có thể hội tụ về điểm thấp cục bộ chứ không phải cực tiểu toàn cục (Global Minimum).
- **Cao nguyên (Plateau):** Hàm chi phí đi ngang, khiến thuật toán mất rất nhiều thời gian để đi qua và có thể bị dừng lại sớm.
- Rất may, hàm chi phí MSE của hồi quy tuyến tính là **hàm lồi (convex function)**, đảm bảo không có cực tiểu cục bộ.



Hình 4-6. Cạm bẫy cực tiểu cục bộ

Tầm quan trọng của việc điều chỉnh đặc trưng (Scaling)

- Khi sử dụng Gradient Descent, bạn phải đảm bảo rằng tất cả các đặc trưng có thang đo (scale) tương tự nhau (VD: dùng StandardScaler).
- **Bên trái:** Các đặc trưng được chia tỷ lệ bằng nhau, hàm chi phí có dạng cái bát tròn, đường đi đến cực tiểu nhanh gọn và thẳng.
- **Bên phải:** Đặc trưng 1 nhỏ hơn Đặc trưng 2, hàm chi phí có dạng bát kéo dài. Đường đi của thuật toán ngoằn ngoèo, tốn nhiều thời gian hội tụ.



Hình 4-7. Gradient Descent với và không có Scaling

Hạ Gradient theo lô (Batch Gradient Descent)

- Batch Gradient Descent (Hạ Gradient theo lô) tính toán đạo hàm riêng của hàm chi phí đối với mỗi tham số θ_j bằng cách sử dụng **toàn bộ tập dữ liệu huấn luyện** tại mỗi bước lặp.
- **Vector Gradient của hàm chi phí:**

$$\nabla_{\theta} MSE(\theta) = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y}) \quad (5)$$

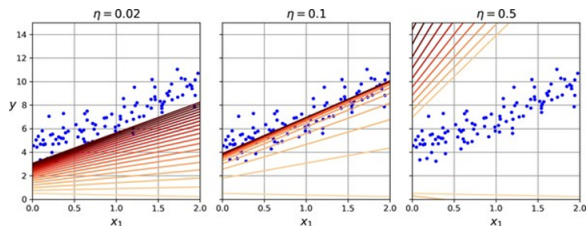
- **Bước nhảy Gradient Descent:**

$$\theta^{(\text{bước tiếp theo})} = \theta - \eta \nabla_{\theta} MSE(\theta) \quad (6)$$

- Mặc dù chậm trên tập dữ liệu rất lớn (do phải dùng toàn bộ dữ liệu), nhưng nó lại chia tỷ lệ tốt với số lượng đặc trưng lớn (tốt hơn Phương trình chuẩn tắc).

Minh họa: Batch GD với các tốc độ học khác nhau

- Hình bên cho thấy 10 bước đầu tiên của Gradient Descent theo lô.
- Trái ($\eta = 0.02$):** Tốc độ học quá thấp, sẽ đạt cực tiểu nhưng chậm.
- Giữa ($\eta = 0.1$):** Tốc độ học rất tốt, thuật toán hội tụ nhanh chóng chỉ trong vài bước.
- Phải ($\eta = 0.5$):** Tốc độ học quá cao, thuật toán nhảy loạn xạ và đang phân kỳ (diverging).



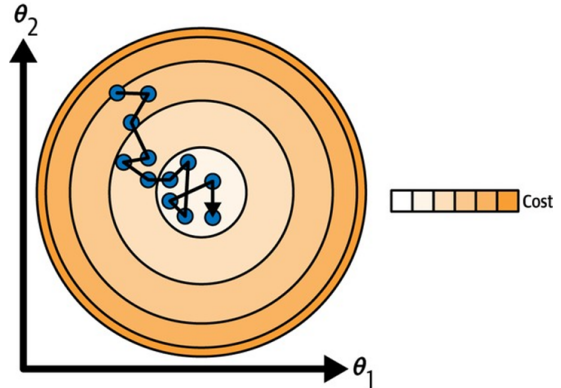
Hình 4-8. Batch GD với tốc độ học khác nhau

Hạ Gradient ngẫu nhiên (Stochastic Gradient Descent - SGD)

- Vấn đề chính của Batch GD là sử dụng toàn bộ tập dữ liệu huấn luyện để tính gradient tại mỗi bước, làm nó rất chậm với tập dữ liệu lớn.
- **Hạ Gradient ngẫu nhiên (SGD)** ngược lại, chọn ra một trường hợp ngẫu nhiên trong tập huấn luyện tại mỗi bước và chỉ tính gradient trên trường hợp duy nhất đó.
- Điều này làm cho thuật toán cực kỳ nhanh, và có khả năng thực hiện huấn luyện các mô hình rất lớn vì mỗi lần lặp chỉ cần 1 trường hợp trong bộ nhớ (hỗ trợ Out-of-core learning).

Đặc điểm đường đi của Stochastic Gradient Descent

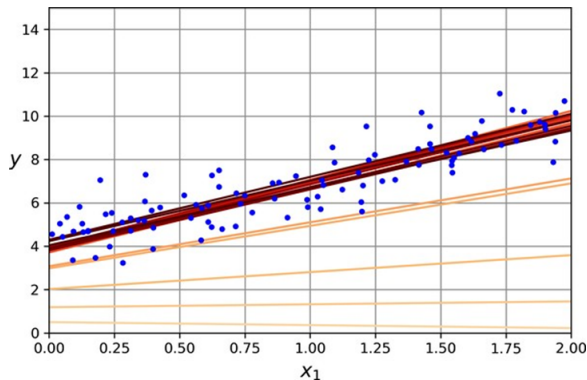
- Do tính chất ngẫu nhiên, SGD thất thường hơn nhiều so với Batch GD.
- Thay vì giảm nhẹ nhàng cho đến khi đạt mức tối thiểu, hàm chi phí sẽ dao động lên xuống, giảm dần về trung bình.
- Ngay cả khi kết thúc gần cực tiểu, các giá trị tham số sẽ tiếp tục dao động. Nghĩa là giá trị cuối cùng là tốt, nhưng chưa phải tối ưu tuyệt đối.
- Ưu điểm: Sự ngẫu nhiên giúp nó nhảy ra khỏi các cực tiểu cục bộ tốt hơn Batch GD.



Hình 4-9. Đặc điểm bất thường của SGD

Minh họa: 20 bước đầu tiên của SGD

- Hình bên cho thấy 20 bước đầu tiên của SGD.
- Bạn có thể thấy các bước không đều đặn như Batch GD. Để SGD hội tụ sát hơn, ta phải áp dụng kỹ thuật **lịch học (learning schedule)**: Tốc độ học lớn lúc đầu (tránh kẹt cực tiểu cục bộ), và giảm dần tốc độ học khi tiến về gần đích để ổn định tại đáy thung lũng.



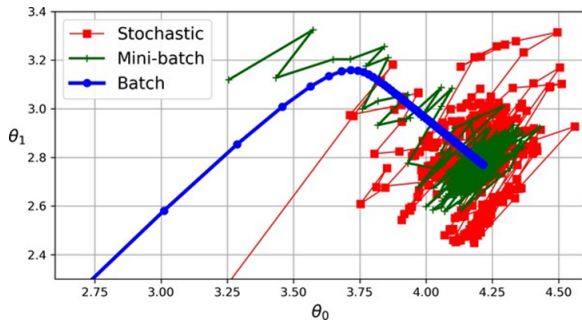
Hình 4-10. 20 bước đầu tiên của SGD

Hạ Gradient theo Mini-Batch

- Tại mỗi bước, thay vì tính toán gradient dựa trên toàn bộ dữ liệu (như Batch GD) hoặc chỉ một trường hợp (như SGD), thuật toán tính gradient trên các tập con nhỏ ngẫu nhiên gọi là **mini-batch**.
- Ưu điểm lớn nhất là bạn có thể tăng hiệu suất từ việc tối ưu hóa phần cứng của các phép toán ma trận, đặc biệt khi sử dụng GPUs.
- Sự tiến bộ của thuật toán ít thất thường hơn SGD (với mini-batch khá lớn). Nó sẽ tiến gần cực tiểu hơn SGD một chút, nhưng có thể khó thoát cực tiểu cục bộ hơn.

So sánh đường đi của 3 thuật toán Gradient Descent

- **Batch GD (Đỏ):** Đi đến thẳng cực tiểu và dừng lại, nhưng tốn nhiều thời gian mỗi bước.
- **SGD (Trắng):** Dao động mạnh xung quanh đích.
- **Mini-Batch GD (Xanh):** Ít dao động hơn SGD và tiến gần đích hơn, nhưng vẫn không dừng lại chính xác tại một điểm trừ khi có lịch học (learning schedule) tốt.



Hình 4-11. Đường đi của 3 thuật toán GD

Bảng so sánh tổng hợp các thuật toán hồi quy tuyến tính

Thuật toán	Lớn m (trường hợp)	Ngoài lõi	Lớn n (đặc trưng)	Hyperparams	Scaling	Scikit-Learn
Phương trình chuẩn tắc	Nhanh	Không	Chậm	0	Không	N/A
SVD	Nhanh	Không	Chậm	0	Không	LinearRegression
Batch GD	Chậm	Không	Nhanh	2	Có	N/A
Stochastic GD	Nhanh	Có	Nhanh	≥ 2	Có	SGDRegressor
Mini-batch GD	Nhanh	Có	Nhanh	≥ 2	Có	N/A

Lưu ý: Sau khi huấn luyện, tất cả các thuật toán này đều tạo ra mô hình rất giống nhau và đưa ra dự đoán theo cùng một cách.

3. Hồi quy đa thức, Đường cong học tập & Chính quy hóa

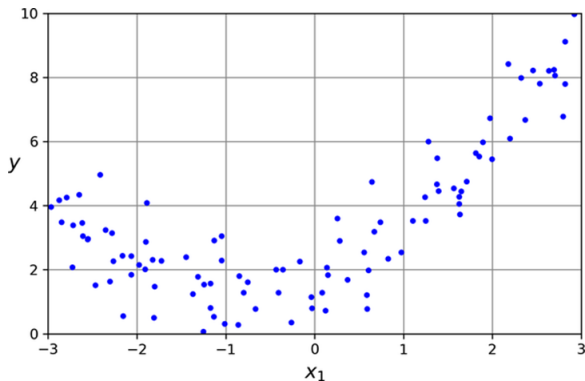
Hồi quy đa thức (Polynomial Regression)

- Điều gì sẽ xảy ra nếu dữ liệu của bạn phức tạp hơn một đường thẳng?
- Thật đáng ngạc nhiên, bạn có thể sử dụng một mô hình tuyến tính để khớp với dữ liệu phi tuyến tính.
- Cách đơn giản: Thêm các lũy thừa của mỗi đặc trưng làm đặc trưng mới, sau đó huấn luyện mô hình tuyến tính trên tập đặc trưng mở rộng này.
- Kỹ thuật này được gọi là **Hồi quy đa thức**.
- `PolynomialFeatures` trong Scikit-Learn có thể dễ dàng tạo ra các biến bậc cao và kết hợp biến (tương tác giữa các đặc trưng).

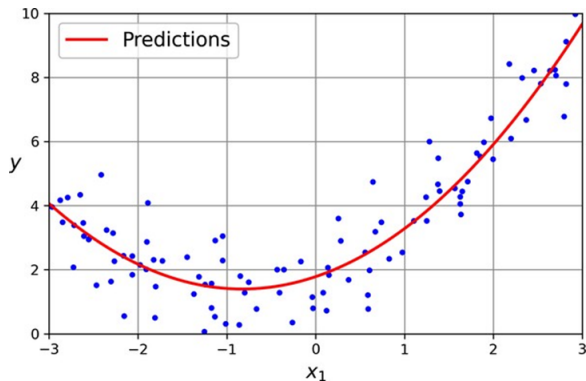
Minh họa: Khớp Hồi quy đa thức trên dữ liệu phi tuyến

- Giả sử chúng ta có một tập dữ liệu phân bố theo phương trình bậc hai ($y = ax^2 + bx + c + \text{nhiều}$). Một đường thẳng tuyến tính bình thường không bao giờ khớp được (Underfitting).
- Bằng cách dùng `PolynomialFeatures(degree=2)`, chúng ta thêm x^2 vào bộ đặc trưng. Sau đó dùng `LinearRegression` khớp dữ liệu mới.
- Kết quả thu được là một đường parabol bám rất sát dữ liệu (như Hình 4-13).

Minh họa: Dữ liệu phi tuyến tính và Dự đoán Hồi quy Đa thức



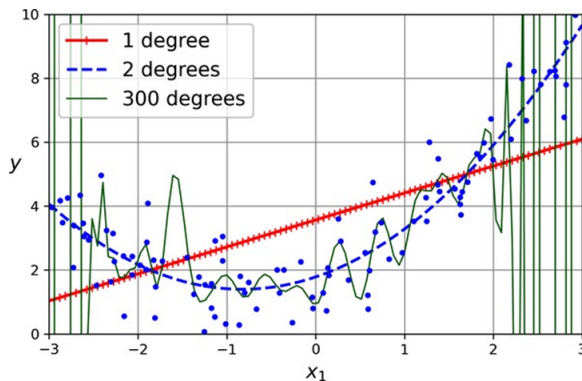
Hình 4-12. Tập dữ liệu phi tuyến tính ngẫu nhiên



Hình 4-13. Dự đoán mô hình hồi quy đa thức

Hiện tượng Quá khớp (Overfitting) với đa thức bậc cao

- Nếu thực hiện hồi quy đa thức bậc quá cao (Ví dụ: bậc 300), mô hình sẽ khớp dữ liệu huấn luyện cực kỳ tốt.
- Hình bên cho thấy đường uốn lượn dữ dội cố gắng đi qua mọi điểm.
- Tuy nhiên, đây là **Quá khớp nghiêm trọng**. Mô hình bậc 2 (Quadratic) mới là mô hình tổng quát hóa tốt nhất (Generalize) cho bộ dữ liệu này.
- Làm sao nhận biết mô hình đang quá khớp hay dưới khớp? Hãy sử dụng Đường cong học tập.

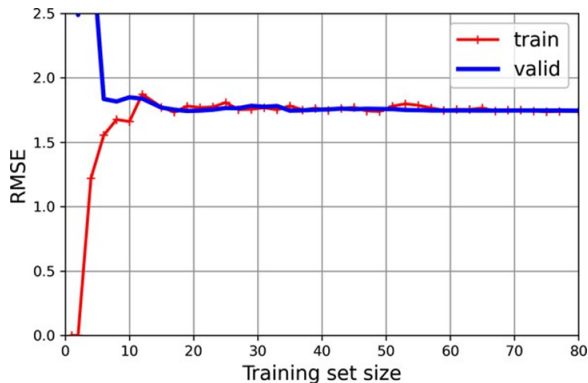


Hình 4-14. Hồi quy đa thức bậc cao (Overfitting)

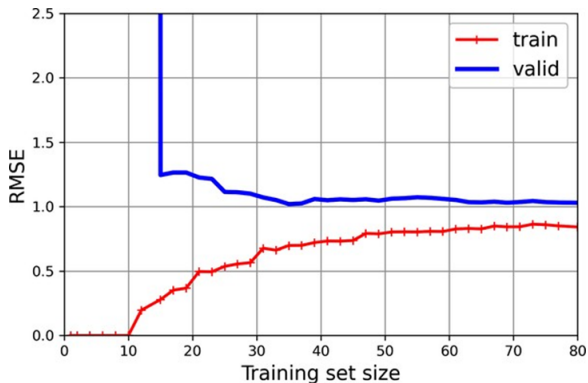
Đánh giá mô hình bằng Đường cong học tập (Learning Curves)

- Đường cong học tập là các biểu đồ thể hiện lỗi trên tập Huấn luyện (Train) và tập Xác thực (Validation) của mô hình dưới dạng hàm của **kích thước tập huấn luyện**.
- **Dưới khớp (Underfitting)**: Cả hai đường đều đạt cao nguyên, lỗi cao và sát nhau (không cải thiện thêm dù cho dữ liệu).
- **Quá khớp (Overfitting)**: Lỗi trên tập huấn luyện thấp hơn nhiều, và có **khoảng cách (gap)** đáng kể giữa lỗi huấn luyện và lỗi xác thực (Mô hình hoạt động rất tốt trên tập huấn luyện nhưng kém trên dữ liệu mới).

Minh họa: Đường cong học tập (Dưới khớp và Quá khớp)



Hình 4-15. Dưới khớp (Đường cong sát nhau, lỗi cao)



Hình 4-16. Quá khớp (Khoảng cách rõ ràng giữa 2 đường cong)

Sự đánh đổi giữa Độ chệch (Bias) và Phương sai (Variance)

- Lỗi tổng quát hóa có thể phân rã thành ba loại lỗi:
- **Độ chệch (Bias):** Lỗi do các giả định sai của mô hình (Vd: giả định dữ liệu tuyến tính khi thực tế là phi tuyến). Độ chệch cao thường gây ra underfitting.
- **Phương sai (Variance):** Lỗi do mô hình quá nhạy cảm với biến động nhỏ trong tập huấn luyện. Một mô hình nhiều bậc tự do (bậc cao) có phương sai cao, gây ra overfitting.
- **Lỗi không thể giảm (Irreducible error):** Do độ nhiễu của bản thân dữ liệu. Chỉ có thể giảm bằng cách làm sạch dữ liệu.
- **Sự đánh đổi:** Tăng độ phức tạp mô hình $\rightarrow$ Tăng variance, giảm bias. Giảm độ phức tạp $\rightarrow$ Tăng bias, giảm variance.

Chính quy hóa (Regularization) Mô hình Tuyến tính

Hồi quy Ridge (Chuẩn hóa Tikhonov)

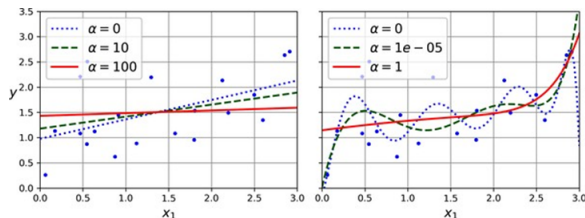
- Một cách tốt để giảm Overfitting là **chính quy hóa (ràng buộc)** mô hình. Đối với mô hình tuyến tính, điều này thường thực hiện bằng cách ràng buộc kích thước của các trọng số θ .
- **Hồi quy Ridge** thêm một số hạng phạt l_2 (chuẩn l_2) vào hàm chi phí MSE:

$$J(\theta) = MSE(\theta) + \alpha \frac{1}{2} \sum_{i=1}^n \theta_i^2 \quad (7)$$

- α kiểm soát mức độ chính quy hóa. Nếu $\alpha = 0$, nó là hồi quy tuyến tính bình thường. Nếu α lớn, tất cả trọng số θ sẽ gần về 0, tạo thành đường thẳng phẳng cắt qua trung bình dữ liệu.

Minh họa: Hồi quy Ridge

- **Bên trái:** Hồi quy Ridge áp dụng trên mô hình tuyến tính thông thường. α tăng làm cho đường dự đoán phẳng hơn, giảm độ chệch của cực đoan.
- **Bên phải:** Hồi quy Ridge áp dụng sau biến đổi PolynomialFeatures(degree=10). Dễ dàng thấy $\alpha = 0$ thì dao động cực mạnh, nhưng α lớn thì đường cong trở nên êm hơn nhiều, giảm overfitting đáng kể.



Hình 4-17. Hồi quy Ridge trên mô hình tuyến tính và đa thức

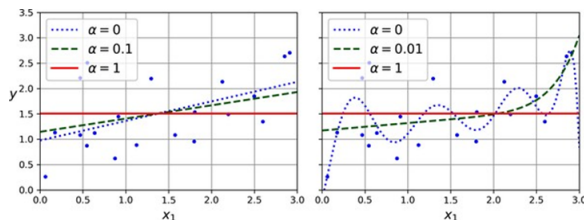
- **Hồi quy Lasso** (Least Absolute Shrinkage and Selection Operator Regression) tương tự Ridge nhưng sử dụng chuẩn l_1 làm số hạng phạt:

$$J(\boldsymbol{\theta}) = MSE(\boldsymbol{\theta}) + \alpha \sum_{i=1}^n |\theta_i| \quad (8)$$

- Điểm đặc biệt nhất của Lasso: Nó có xu hướng hoàn toàn loại bỏ (đặt trọng số bằng 0) đối với các đặc trưng ít quan trọng nhất.
- Do đó, Lasso tự động thực hiện **lựa chọn đặc trưng (feature selection)**, tạo ra một mô hình thưa (sparse model).

Minh họa: Hồi quy Lasso

- Hình bên minh họa Lasso trên dữ liệu tuyến tính (trái) và đa thức (phải).
- Với $\alpha = 1$, đối với mô hình đa thức, đường dự đoán gần như trở thành một đường thẳng bậc thấp, chứng tỏ các tham số bậc cao đã bị đưa về giá trị 0 hoàn toàn.

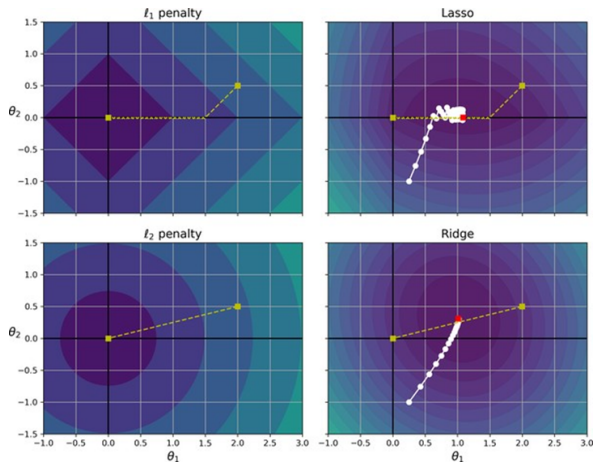


Hình 4-18. Hồi quy Lasso trên mô hình tuyến tính và đa thức

Điểm khác biệt giữa Lasso và Ridge trong không gian tham số

- Ridge (chuẩn l_2) khiến các trọng số giảm dần khi tiến tới gốc tọa độ, gradient trở nên nhỏ nên việc đi đến tối ưu hội tụ êm và không dao động. Các tham số không bao giờ bị loại bỏ hoàn toàn (không về 0).
- Lasso (chuẩn l_1) thì hàm chi phí giảm tuyến tính khi tiến tới các trục (trục θ_1, θ_2). Hạ gradient trên Lasso sẽ nhanh chóng khiến một tham số về 0 (như $\theta_2 = 0$), sau đó trượt trên trục θ_1 cho đến khi cả hai đạt cực tiểu.
- Kết quả: Lasso tạo ra mô hình thưa và loại bỏ đặc trưng.

Minh họa: Lasso vs Ridge trong Không gian tham số



Hình 4-19. Chính quy hóa Lasso (Trên) và Ridge (Dưới)

Hồi quy Elastic Net & Khi nào nên dùng?

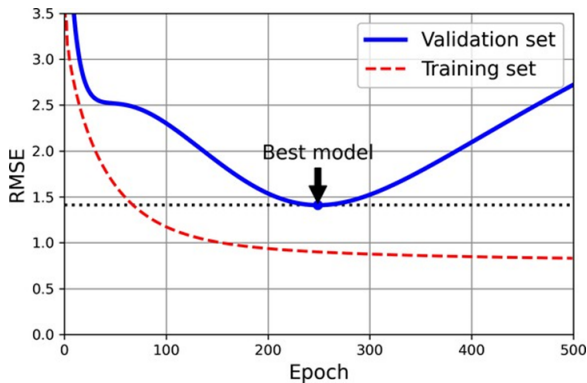
- **Elastic Net** là sự kết hợp trung gian giữa Ridge và Lasso. Hàm phạt của nó kết hợp cả chuẩn l_1 và l_2 , được điều khiển bởi tỷ lệ trộn r .
- Khi $r = 0$, nó là Ridge; Khi $r = 1$, nó là Lasso.
- **Khi nào sử dụng loại nào?**
 - Luôn ưu tiên dùng một ít chính quy hóa. **Ridge là lựa chọn mặc định tốt.**
 - Nếu bạn nghi ngờ chỉ có một vài đặc trưng là hữu ích (Nhiều nhiều), dùng **Lasso hoặc Elastic Net** để loại bỏ các đặc trưng vô ích.
 - Nói chung, **Elastic Net được ưa chuộng hơn Lasso** vì Lasso có thể hoạt động thất thường khi số lượng đặc trưng lớn hơn số trường hợp, hoặc khi các đặc trưng có tương quan mạnh với nhau.

Dừng sớm (Early Stopping)

- Một cách rất khác (và đơn giản) để chính quy hóa các thuật toán học lặp (như GD) là dừng huấn luyện ngay khi Lỗi xác thực (Validation Error) đạt đến cực tiểu.
- Khi các epoch trôi qua, lỗi trên tập huấn luyện luôn giảm. Lỗi xác thực cũng giảm ban đầu, nhưng sau đó sẽ tăng ngược trở lại (Đó là lúc mô hình bắt đầu Overfitting).
- Chỉ cần dừng thuật toán ngay tại điểm lỗi xác thực chạm mức thấp nhất. Kỹ thuật này hiệu quả đến mức Geoffrey Hinton gọi nó là **"bữa trưa miễn phí tuyệt vời"**.

Minh họa: Chính quy hóa bằng Dừng sớm (Early Stopping)

- Mũi tên chỉ vào thời điểm tốt nhất để dừng huấn luyện.
- Các bước sau đó chỉ làm mô hình tệ hơn khi đối mặt với dữ liệu chưa từng thấy (Mặc dù lỗi RMSE trên tập huấn luyện vẫn đang giảm).



Hình 4-20. Early Stopping

4. Hồi quy Logistic & Softmax

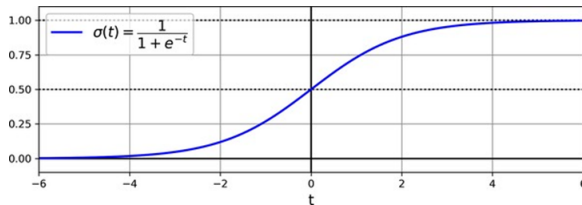
Hồi quy Logistic (Logistic Regression)

- Dù mang tên "Hồi quy", đây lại là mô hình thường được sử dụng cho **Phân loại** (Thường là nhị phân - 2 lớp).
- Mô hình ước tính xác suất một trường hợp thuộc về một lớp cụ thể (Vd: 70% là thư rác).
- Nếu xác suất ước tính $> 50\%$, nó dự đoán thuộc lớp "1" (dương). Ngược lại là lớp "0" (âm).
- Hoạt động giống Hồi quy tuyến tính, tính tổng trọng số các đặc trưng. Nhưng thay vì xuất kết quả trực tiếp, nó xuất ra **hàm logistic** của kết quả đó.

$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^T \mathbf{x}) \quad (9)$$

Hàm Logistic (Sigmoid)

- Hàm logistic (σ) là một hàm sigmoid (có hình chữ S) lấy đầu vào là bất kỳ số thực nào và trả về kết quả trong khoảng từ 0 đến 1.
- $\sigma(t) = \frac{1}{1+\exp(-t)}$
- Biểu đồ bên cho thấy giá trị $\sigma(t)$ tiến về 1 khi t lớn và tiến về 0 khi t là số âm lớn.



Hình 4-21. Đồ thị Hàm Logistic (Sigmoid)

Hàm huấn luyện và Hàm chi phí (Log Loss)

- Hàm chi phí đối với một trường hợp huấn luyện đơn lẻ:

$$c(\theta) = \begin{cases} -\log(\hat{p}) & \text{nếu } y = 1 \\ -\log(1 - \hat{p}) & \text{nếu } y = 0 \end{cases} \quad (10)$$

- Có ý nghĩa: $-\log(t)$ rất lớn khi t tiến tới 0. Lỗi rất lớn nếu mô hình dự đoán xác suất gần 0 cho lớp thực sự là 1, và ngược lại.
- Hàm chi phí trên toàn bộ tập (Log Loss):**

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right] \quad (11)$$

- Không có phương trình dạng đóng để tính trực tiếp (như PT Chuẩn tắc), nhưng Log Loss là hàm lồi nên có thể dùng **Gradient Descent** để tối ưu.

Tập dữ liệu hoa Diên vĩ (Iris Dataset)

- Để minh họa trực quan Logistic Regression, chúng ta dùng tập dữ liệu nổi tiếng Iris.
- Gồm thông số chiều dài/chiều rộng đài hoa và cánh hoa của 150 bông diên vĩ thuộc 3 loài: *Setosa*, *Versicolor*, *Virginica*.
- Bài toán: Phân loại một bông hoa có phải loài *Virginica* hay không dựa trên chiều rộng cánh hoa.

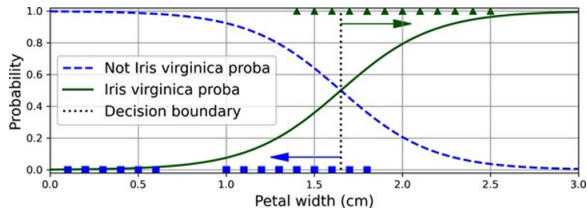


Hình 4-22. Ba loài hoa Diên vĩ

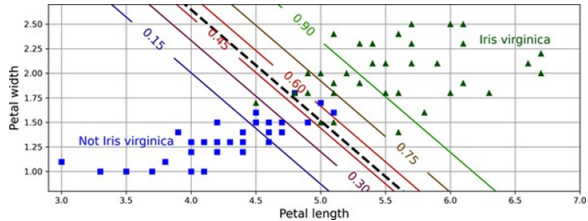
Các đường ranh giới quyết định (Decision Boundaries)

- Khi mô hình đã được huấn luyện, chúng ta xem các xác suất ước tính của mô hình cho các bông hoa.
- Ranh giới quyết định là điểm mà tại đó mô hình thay đổi quyết định (xác suất = 50%).
- Mô hình Hồi quy Logistic có ranh giới quyết định là tuyến tính. Nếu có 1 đặc trưng, nó là 1 điểm; có 2 đặc trưng, nó là 1 đường thẳng; > 2 đặc trưng, nó là một siêu mặt phẳng.

Minh họa: Ranh giới quyết định Hồi quy Logistic



Hình 4-23. Ranh giới quyết định (1 chiều - chiều rộng cánh hoa)



Hình 4-24. Đường ranh giới tuyến tính (2 chiều - rộng & dài)

Hồi quy Softmax (Hồi quy Logistic Đa thức)

- Hồi quy Logistic có thể tổng quát hóa hỗ trợ trực tiếp đa lớp mà không cần phải kết hợp nhiều bộ phân loại nhị phân (OvR). Được gọi là **Hồi quy Softmax**.
- Cách thức:
 - Tính điểm số $s_k(\mathbf{x})$ cho từng lớp k : $s_k(\mathbf{x}) = (\boldsymbol{\theta}^{(k)})^\top \mathbf{x}$
 - Biến đổi điểm số thành xác suất ước tính $\hat{p}_k$ thông qua **Hàm Softmax**.
- Hàm Softmax tính toán cấp số nhân (hàm mũ) của mọi điểm số rồi chuẩn hóa chúng.

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))} \quad (12)$$

Dự đoán và Hàm chi phí Cross Entropy

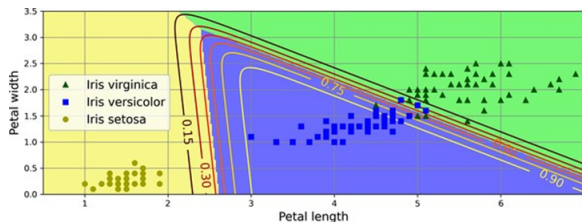
- Mô hình Softmax sẽ dự đoán lớp có xác suất cao nhất.
- **Hàm chi phí Cross Entropy (Mất mát chéo):**

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)}) \quad (13)$$

- Cross entropy rất hiệu quả vì nó phạt nặng các mô hình khi ước lượng một xác suất thấp cho lớp đúng (mục tiêu).
- Khi $K = 2$ (2 lớp), Cross Entropy tương đương chính xác với hàm Log Loss của Logistic Regression.

Minh họa: Ranh giới quyết định của Hồi quy Softmax

- Chạy Softmax trên cả 3 lớp hoa Iris.
- Hình bên cho thấy ranh giới phân định giữa 3 lớp.
- Các ranh giới quyết định giữa bất kỳ hai lớp nào đều là **tuyến tính** (Đường thẳng).
- Tại trung tâm, ranh giới 3 lớp giao nhau, xác suất các lớp đều bằng 33%.



Hình 4-25. Ranh giới quyết định hồi quy Softmax

Tổng kết Chương 4

Tổng kết Chương 4

- Chúng ta đã khám phá bên trong "Hộp đen" của Hồi quy tuyến tính và giải bằng Phương trình chuẩn tắc, SVD.
- Hiểu rõ và áp dụng thuật toán tối ưu mạnh mẽ là **Gradient Descent** dưới nhiều hình thức (Batch, SGD, Mini-batch) cũng như tầm quan trọng của tốc độ học (Learning rate).
- Mở rộng khả năng của mô hình cho dữ liệu phức tạp bằng **Hồi quy đa thức**.
- Biết cách sử dụng **Đường cong học tập** để chẩn đoán Underfitting/Overfitting.
- Triển khai các phương pháp **Chính quy hóa** (Ridge, Lasso, Elastic Net, Early Stopping) để ngăn chặn Overfitting.
- Làm quen với các công cụ Phân loại xác suất thông qua **Hồi quy Logistic** và **Hồi quy Softmax**.